PROJECT DEMONSTRATION GUIDE

Metro Ticket Generating System in ServiceNow

Prepared By	Devi k
Platform	ServiceNow
Document Type	Final Project Demonstration
Demo Scope	Booking to QR Validation
Date	29 September 2026

1. DEMONSTRATION PURPOSE

This document defines the final demonstration flow for the Metro Ticket Generating System in ServiceNow. The demonstration is intended to show the complete user journey from opening the ServiceNow portal and booking a metro ticket through fare processing, Flow Designer execution, database storage, QR-based ticket presentation, request tracking, and station-side verification.

- Demonstrate the passenger-facing booking experience.
- Show the automated ServiceNow workflow behind the request.
- Prove that booking data is stored in the custom Metro Database table.
- Show QR ticket generation and Service Portal-compatible display.
- Demonstrate requester and operational/fulfiller perspectives.
- Explain the major issues faced during development and how they were resolved.

2. DEMONSTRATION ARCHITECTURE

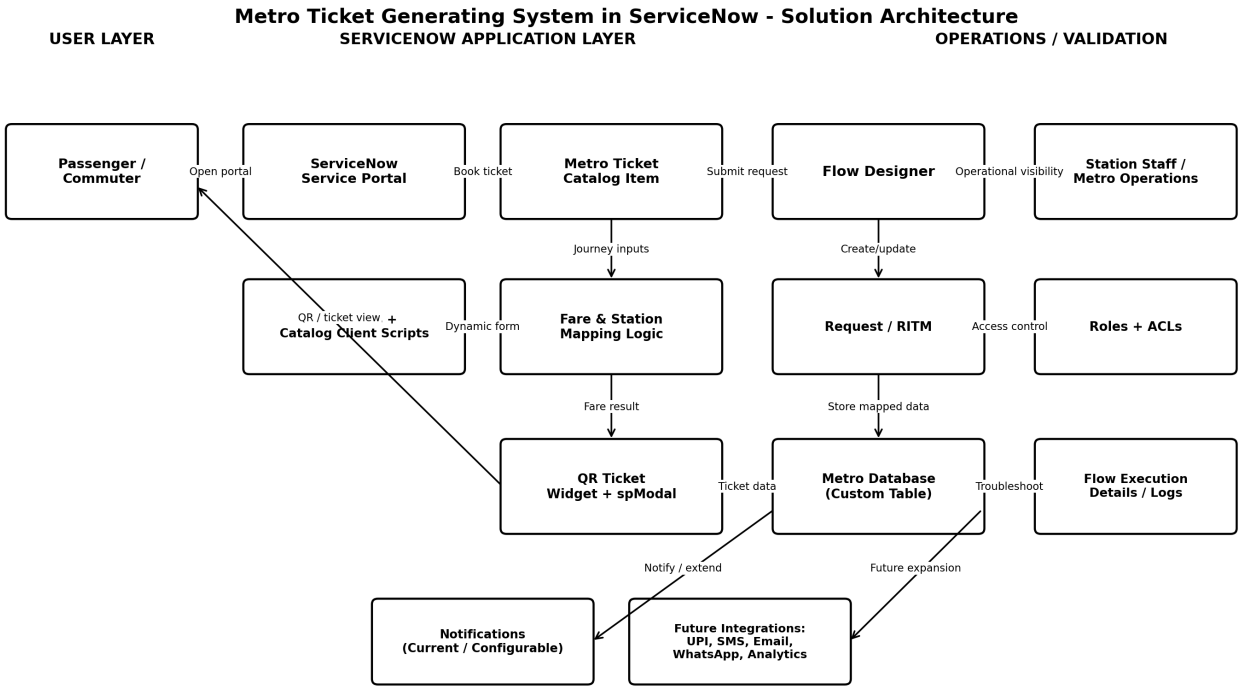


Figure 1. End-to-end solution architecture used during the demonstration

The live demo should follow the same architectural path shown above: Passenger -> Service Portal -> Metro Ticket Catalog Item -> fare/station logic -> Flow Designer -> Request/RITM and Metro Database -> QR ticket widget/spModal -> station or operations validation.

3. DEMO PREREQUISITES

Requirement	Demo Readiness Check
ServiceNow Instance	PDI or authorized project instance is active and accessible.
Requester Account	A user that can open the portal and submit the metro catalog item.
Operations/Admin Account	Authorized account for Flow Designer, custom table and ACL/record validation.
Catalog Item	Metro ticket booking item is published and accessible.
Sample Journey	Prepare one known source-destination pair with an expected fare.
QR Feature	Widget/spModal QR view is enabled and ready.
Browser	Use a clean browser session; keep requester and admin views in separate tabs or profiles.

4. FINAL LIVE DEMONSTRATION FLOW

Step	Demo Action	Role	What to Show	Expected Result
Step 1	Open ServiceNow Service Portal	Requester	Open the portal home page and navigate to the metro service/catalog category.	Portal loads and the metro booking service is visible.
Step 2	Open “Book A Metro Service”	Requester	Open the Metro Ticket Catalog Item. Point out Source, Destination, Passenger Type and Number of Tickets.	All configured variables are visible and usable.
Step 3	Enter Journey Details	Requester	Select a prepared Source and Destination, choose Passenger Type, and enter the ticket quantity.	Dynamic form behaviour works; dependent values remain consistent.
Step 4	Demonstrate Fare Logic	Requester/System	Trigger the fare calculation and compare the result with the prepared expected fare.	Correct fare is displayed/processed for the selected route and quantity.
Step 5	Submit the Booking	Requester	Submit the catalog request. Show request confirmation/reference.	REQ/RITM is created and Flow Designer starts processing.
Step 6	Show Flow Designer Execution	Admin / Fulfiller	Open Flow Designer Execution Details for the submitted request and show the completed actions/condition path.	Expected actions complete without unresolved failure.
Step 7	Show Metro Database Record	Admin / Operations	Open the custom Metro Database table and locate the record created from the booking.	Source, Destination, Passenger Type, quantity, fare and ticket/request data are mapped correctly.
Step 8	Open the QR Digital Ticket	Requester	Return to the passenger view and open the generated digital ticket / QR display.	QR/ticket view renders correctly in Service Portal.
Step 9	Track Request Status	Requester	Open the request/status view and show the booking status.	Passenger can identify the current request state.
Step 10	Station / Operations Verification	Station Staff / Operations	Switch to the authorized operational view and verify the ticket/booking details against the store record.	Authorized user can access the relevant data and validate the ticket.

4.1 Suggested Narration During the Live Demo

Step 1 - Open ServiceNow Service Portal: “I will begin from the passenger side. The passenger accesses the ServiceNow portal and selects the metro ticket booking service.”

Step 2 - Open “Book A Metro Service”: “The booking form captures only the information needed for the journey, keeping the user flow simple.”

Step 3 - Enter Journey Details: “UI Policies and essential Catalog Client Scripts control validation and dynamic form behaviour.”

Step 4 - Demonstrate Fare Logic: “The system calculates the fare using station/fare mapping logic, reducing manual fare calculation.”

Step 5 - Submit the Booking: “Once submitted, the booking becomes a ServiceNow request and the automation takes over.”

Step 6 - Show Flow Designer Execution: “Execution Details gives us traceability. We used this same view during development to identify failed steps and incorrect paths.”

Step 7 - Show Metro Database Record: “The catalog request is not only displayed to the user; the relevant data is stored in a structured custom table for operations and verification.”

Step 8 - Open the QR Digital Ticket: “The QR experience uses a Service Portal-compatible widget and spModal approach because the original portal context caused rendering issues.”

Step 9 - Track Request Status: “The passenger retains visibility after submission instead of losing track of the booking.”

Step 10 - Station / Operations Verification: “This completes the journey from passenger booking to operational verification.”

5. ROLE-BASED DEMONSTRATION

Role	Key Demo Activities	Main Components
Passenger / Requester	Book ticket; enter journey details; review fare; submit request; track status; open QR ticket.	Service Portal, Service Catalog, request status, QR view.
Station Staff / Metro Operations	Verify ticket and booking information using authorized operational access.	Ticket/booking record and validation data.
ServiceNow Administrator	Configure and demonstrate Flow Designer, custom table, roles/ACLs, troubleshooting and execution logs.	Flow Designer, Execution Details, tables, roles, ACLs.

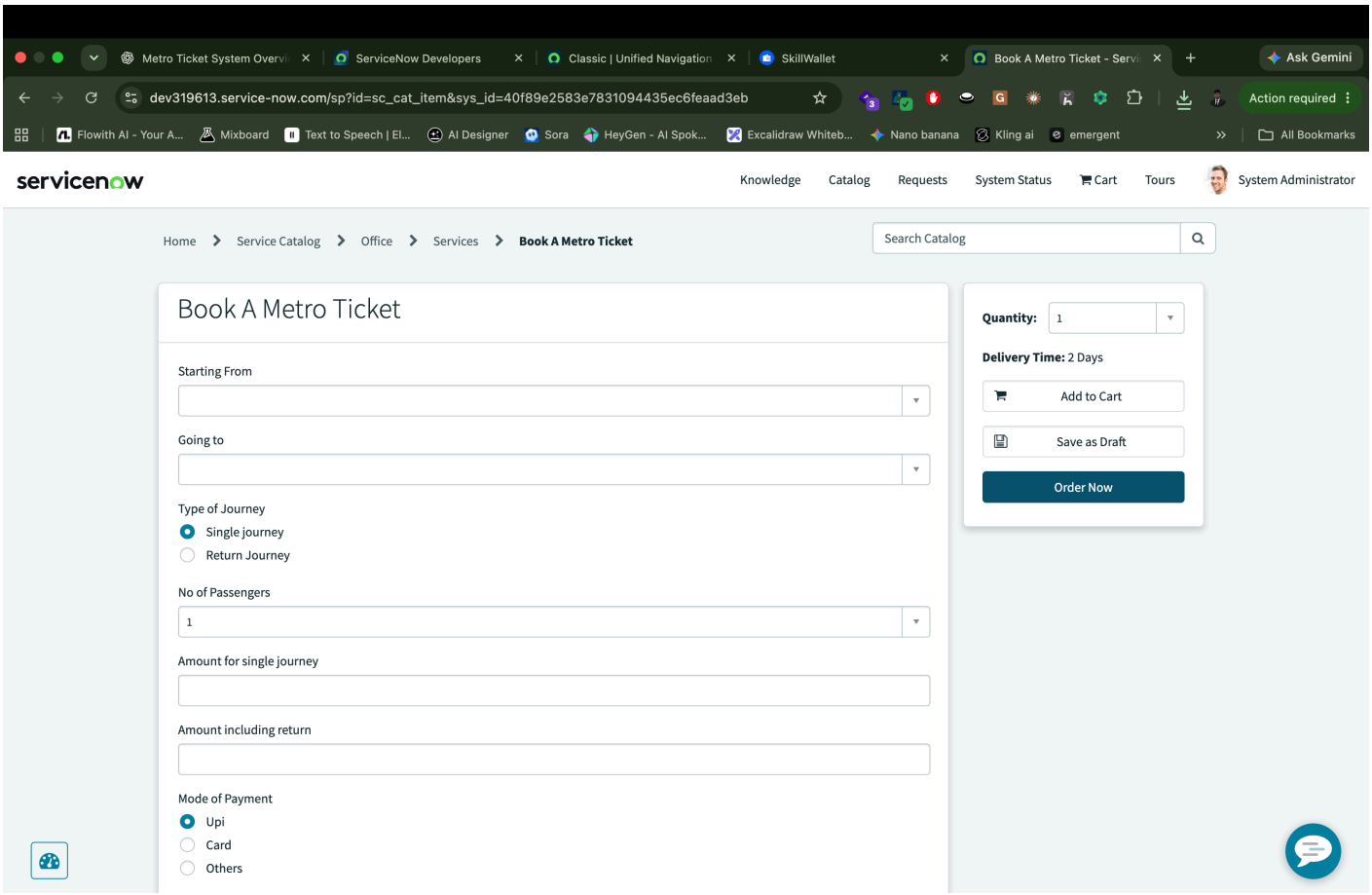
6. PROBLEMS FACED AND LIVE EXPLANATION

Problem	What Happened	Resolution	How to Demonstrate
QR rendering in Service Portal	QR behaviour was inconsistent because the portal execution context/sandbox differed from backend behaviour.	Adopted a portal-compatible widget and spModal architecture.	Show the working QR view and explain why the architecture changed.
Fare mismatch	Some routes produced unexpected fares during testing.	Used logs/testing to inspect route inputs and refined the fare/station mapping logic.	Run a known route and show the correct fare.
Variable-to-field mapping	Some submitted catalog values were missing or mapped incorrectly in the custom table.	Corrected bindings/data pills and required client-side population.	Compare the submitted variables with the Metro Database record.
Flow execution failures	Certain actions/conditions failed or followed the wrong path during configuration.	Used Flow Designer Execution Details to identify the failed action and verify the corrected path.	Open a successful flow execution and point out the action sequence.
Portal form behaviour	Visibility/population rules needed portal-specific verification.	Tested UI Policies and minimal Catalog Client Scripts in preview and Service Portal.	Change a field selection and show the expected form behaviour.

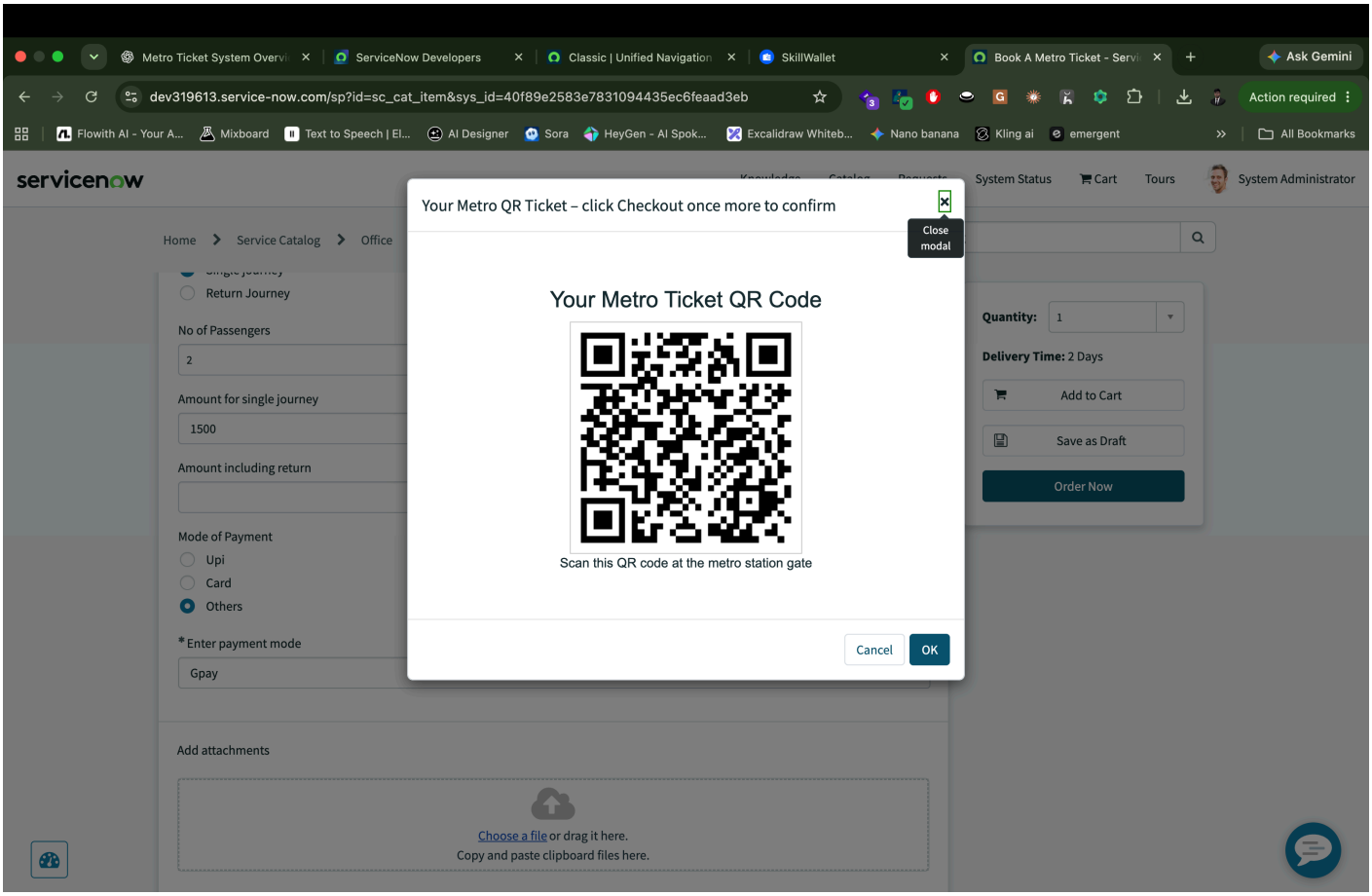
7. DEMONSTRATION EVIDENCE / SCREENSHOTS

Use the following evidence sequence in the final submission or presentation. Only the project workspace screenshot was supplied in the current documentation set; the remaining screenshots should be captured directly from the working ServiceNow instance during or before the final demonstration.

- Figure 2: Service Portal home / metro service category
- Figure 3: Metro Ticket Catalog Item with variables
- Figure 4: Completed journey inputs and fare
- Figure 5: Request submission confirmation / RITM
- Figure 6: Flow Designer Execution Details



- Figure 7: Metro Database record with mapped values
- Figure 8: QR ticket displayed in Service Portal



• Figure 9: Request status tracking view
• Figure 10: Role/ACL or authorized operational verification view

Available project screenshot: SkillWallet / project workspace showing project objectives and progress

8. DEMO VALIDATION CHECKLIST

No.	Validation Item	Status
1	Catalog item opens without error.	☐ Pass ☐ Fail
2	Required booking variables are visible.	☐ Pass ☐ Fail
3	Source and destination selections are valid.	☐ Pass ☐ Fail
4	Fare result matches prepared expected value.	☐ Pass ☐ Fail
5	Request/RITM is created successfully.	☐ Pass ☐ Fail
6	Flow Designer execution completes expected path.	☐ Pass ☐ Fail
7	Metro Database record contains correct mapped values.	☐ Pass ☐ Fail
8	QR ticket renders in Service Portal.	☐ Pass ☐ Fail
9	Request status is visible to requester.	☐ Pass ☐ Fail
10	Operational role can verify required ticket data.	☐ Pass ☐ Fail
11	No unauthorized data is exposed during role switch.	☐ Pass ☐ Fail
12	No unresolved defect blocks the end-to-end demo.	☐ Pass ☐ Fail

9. EVALUATOR QUESTIONS - QUICK ANSWERS

Question	Answer
Why ServiceNow?	It provides Service Catalog, Flow Designer, custom tables, portal capabilities, security and request tracking in one low-code platform, which fits the goal of a maintainable digital ticketing workflow.
Why use Flow Designer?	It keeps the automation visible and maintainable for administrators and reduces dependence on custom server-side code.
Where is ticket data stored?	Request/RITM preserves the service-request lifecycle, while the custom Metro Database table stores structured metro booking/ticket information.
What was the hardest problem?	QR ticket rendering in Service Portal was one of the most significant issues because portal behaviour differed from the backend context; the final design used widget + spModal.
How is security handled?	ServiceNow authentication is combined with roles and ACLs to restrict operational data and actions.
Is payment implemented?	No. Payment gateway/UPI integration is future scope and is not presented as completed functionality.
Is AI or ML used?	No trained AI/ML model is part of the current implementation. The solution is rule-based and workflow-driven.
How can it scale?	Additional stations, fare bands, passenger categories, notification channels, payments and analytics can be added without replacing the core ServiceNow workflow.

10. FINAL DEMONSTRATION CONCLUSION

The final demonstration should prove that the project is an end-to-end ServiceNow metro ticketing prototype rather than only a static catalog form. The success criteria are that a passenger can submit a valid journey, the system can process and store the booking, the digital QR ticket can be displayed reliably, the request can be tracked, and authorized metro personnel can verify the relevant ticket information. The demonstration should also explain the real implementation challenges and the debugging methods used to reach the final working state.

11. DEMO SIGN-OFF

Demonstrated By	Devi K
Demo Date	_____
Evaluator / Reviewer	_____
Overall Demo Status	☐ Successful ☐ Needs Rework
Signature	_____